

3-编码器知识蒸馏

笔记本: LLM_practice

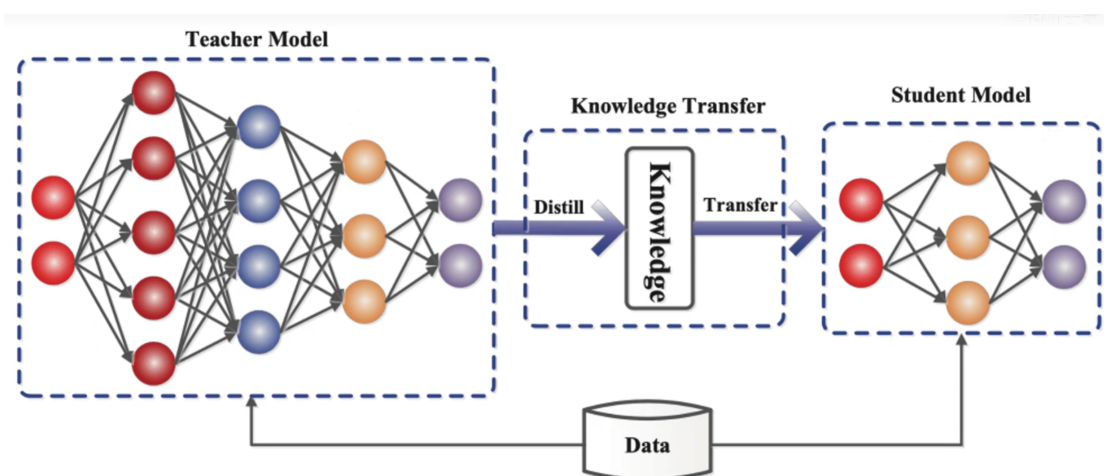
创建时间: 2025/2/2 12:07

更新时间: 2025/3/31 8:11

作者: alexritchie@163.com

URL: https://github.com/wyf3/llm_related/tree/main/knowledge_distillation_llm

3-编码器知识蒸馏



知识蒸馏就是把一个大的教师模型的知识萃取出来，把他浓缩到一个小的学生模型，可以理解为一个大的教师神经网络把他的知识教给小的学生网络，这里有一个知识的迁移过程，从教师网络迁移到了学生网络身上，教师网络一般是比较臃肿，所以教师网络把知识教给学生网络，学生网络是一个比较小的网络，这样就可以用学生网络去做一些轻量化网络做的事情。

一、轻量化模型有哪些方式？

- 压缩已训练好的模型：知识蒸馏、权值量化、权重剪枝、通道剪枝、早退推理
- 直接训练轻量化网络：SqueezeNet、MobileNetv1v2v3、MnasNet、SHuffleNet、Xception、EfficientNet、EfficientDet
- 加速卷积运算：im2col+GEMM、Winograd、低秩分解
- 硬件部署：TensorRT、Jetson、TensorFlow-Slim、openvino、FPGA集成电路

二、为什么要知识蒸馏

深度学习在计算机视觉、语音识别、自然语言处理等内的众多领域中均取得了令人难以置信的性能。但是，大多数模型在计算上过于昂贵，无法在移动端或嵌入式设备上运行。因此需要对模型进行压缩，这样小的模型就适用于部署在终端设备上了。

2.1 提升模型精度

如果对目前的网络模型A的精度不是很满意，那么可以先训练一个更高精度的teacher模型B（通常参数量更多，时延更大），然后用这个训练好的teacher模型B对student模型A进行知识蒸馏，得到一个更高精度的A模型。

例如，使用满血版大模型对小模型进行蒸馏。

2.2 降低模型时延，压缩网络参数

如果对目前的网络模型A的时延不满意，可以先找到一个时延更低，参数量更小的模型B，通常来讲，这种模型精度也会比较低，然后通过训练一个更高精度的teacher模型C来对这个参数量小的模型B进行知识蒸馏，使得该模型B的精度接近最原始的模型A，从而达到降低时延的目的。

2.3 标签之间的域迁移

假如使用狗和猫的数据集训练了一个teacher模型A，使用香蕉和苹果训练了一个teacher模型B，那么就可以用这两个模型同时蒸馏出一个可以识别狗、猫、香蕉以及苹果的模型，将两个不同域的数据集进行集成和迁移。

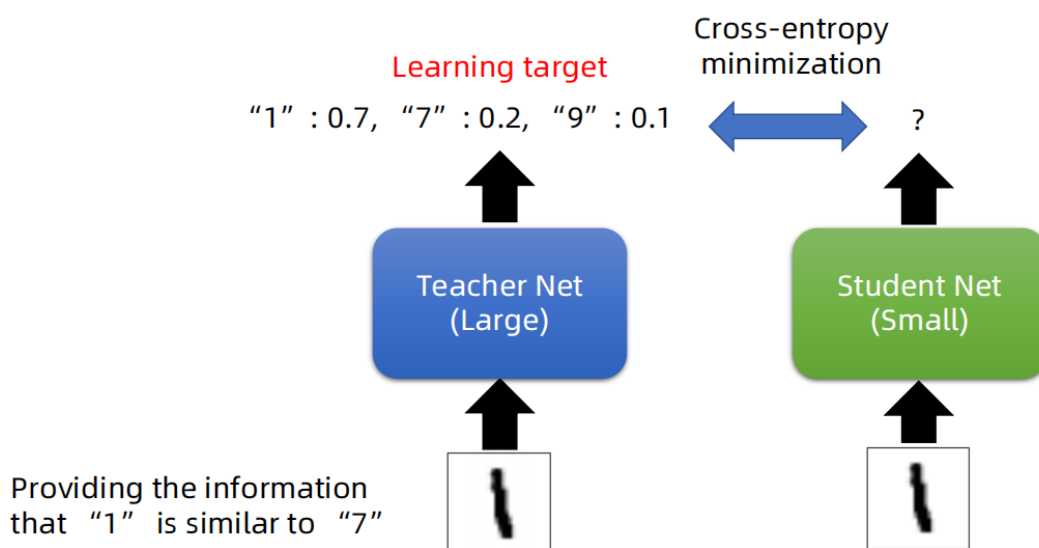
三、知识蒸馏的理论依据

知识蒸馏使用的是Teacher—Student模型，其中teacher是“知识”的输出者，student是“知识”的接受者。知识蒸馏的过程分为2个阶段：

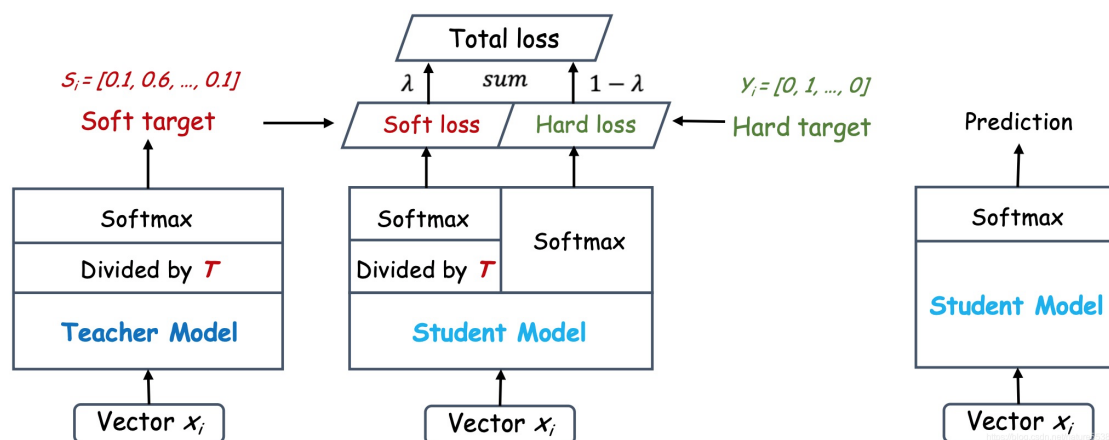
- 原始模型训练：训练"Teacher模型"，简称为Net-T，它的特点是模型相对复杂，也可以由多个分别训练的模型集成而成。我们对"Teacher模型"不作任何关于模型架构、参数量、是否集成方面的限制，唯一的要求就是，对于输入X，其都能输出Y，其中Y经过softmax的映射，输出值对应相应类别的概率值。
- 精简模型训练：训练"Student模型"，简称为Net-S，它是参数量较小、模型结构相对简单的单模型。同样的，对于输入X，其都能输出Y，Y经过softmax映射后同样能输出对应相应类别的概率值。
- Teacher学习能力强，可以将它学到的知识迁移给学习能力相对弱的Student模型，以此来增强Student模型的泛化能力。复杂笨重但是效果好的Teacher模型不上线，就单纯是个导师角色，真正部署上线进行预测任务的是灵活轻巧的Student小模型。

实际，知识蒸馏的主要功能来源于**软标签学习**。这与对比学习中的软标签对比有提升，原理是一样的。软便签计算KL散度，可以在cross-entropy的正确类拟合中，去最小化错误类

的误差。



四、知识蒸馏的过程



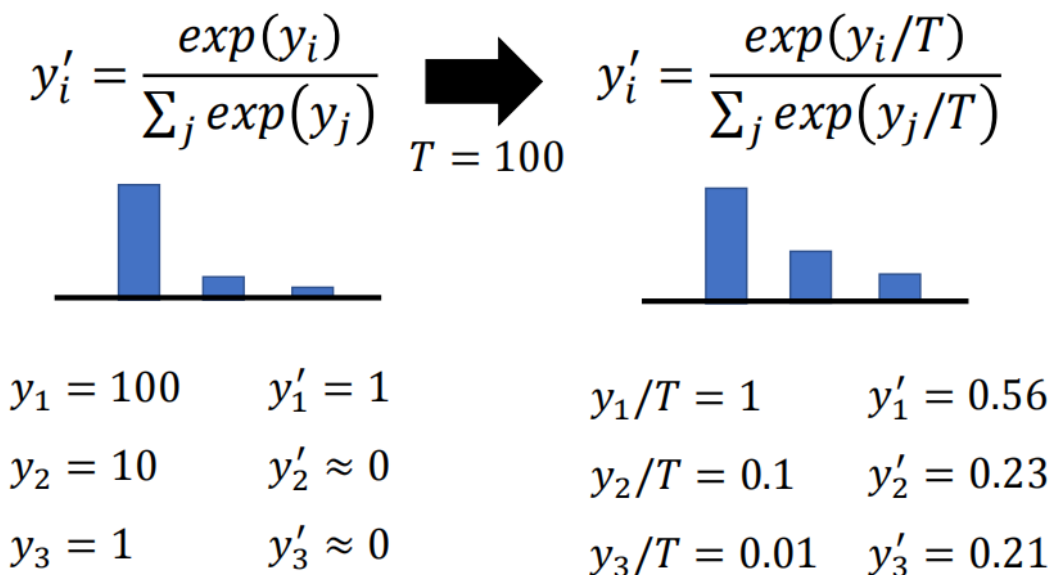
如上图所示，教师网络（左侧）的预测输出除以温度参数（Temperature）之后，再做 Softmax 计算，可以获得软化的概率分布（软目标或软标签），数值介于 0 – 1 之间，取值分布较为缓和。Temperature 数值越大，分布越缓和；而 Temperature 数值减小，容易放大错误分类的概率，引入不必要的噪声。针对较困难的分类或检测任务，Temperature 通常取 1，确保教师网络中正确预测的贡献。硬目标则是样本的真实标注，可以用 One-hot 矢量表示。Total loss 设计为软目标与硬目标所对应的交叉熵的加权平均（表示为 KD loss 与 CE loss），其中软目标交叉熵的加权系数越大，表明迁移诱导越依赖教师网络的贡献，这对训练初期阶段是很有必要的，有助于让学生网络更轻松的鉴别简单样本，但训练后期需要适当减小软目标的比重，让真实标注帮助鉴别困难样本。另外，教师网络的预测精度通常要优于学生网络，而模型容量则无具体限制，且教师网络推理精度越高，越有利于学生网络的学习。

教师网络与学生网络也可以联合训练，此时教师网络的暗知识及学习方式都会影响学生网络的学习，具体如下（式中三项分别为教师网络 Softmax 输出的交叉熵 loss、学生网络

Softmax输出的交叉熵loss、以及教师网络数值输出与学生网络Softmax输出的交叉熵loss)

五、温度的意义

温度可以使标签软化。



左边是正常的softmax计算，右边引入一个超参数T来计算softmax，即把 y_i 都除以T在进行softmax计算。结果会有什么不同呢？

显而易见，当 y_1 远大于 y_2 和 y_3 时，在进行softmax归一化时，其结果会无限趋近于1，反之， y_2 和 y_3 约等于0。如何进行了升温操作，给一个较大的T，这样 y_1 、 y_2 和 y_3 在softmax归一化之前会相对接近一些，如此，得到的结果就会比较接近。

很多场景中，如果需要将标签软化，通常都会以除以较大的T来实现升温操作。

六、使用温度的一般感受

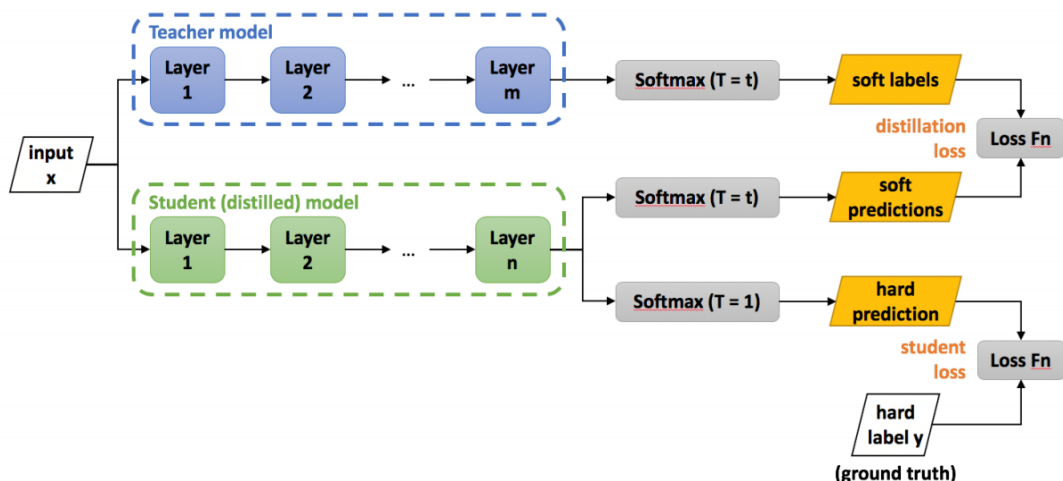
- 原始的softmax函数是 $T=1$ 时的特例； $T<1$ 时，概率分布比原始更“陡峭”，也就是说，当 $T>0$ 时，Softmax的输出值会接近于Hard-target； $T>1$ 时，概率分布比原始更“平缓”。
- 随着T的增加，Softmax的输出分布越来越平缓，信息熵会越来越大。温度越高，softmax上各个值的分布就越平均，思考极端情况，当，此时softmax的值是平均分布的。
- 不管温度T怎么取值，Soft-target都有忽略相对较小的（Teacher模型在温度为T时softmax输出在第i类上的值）携带的信息的倾向。
- 温度的高低改变的是Student模型训练过程中对负标签的关注程度。当温度较低时，对负标签的关注，尤其是那些显著低于平均值的负标签的关注较少；而温度较高时，负标签相关的值会相对增大，Student模型会相对更多地关注到负标签。

- 实际上，负标签中包含一定的信息，尤其是那些负标签概率值显著高于平均值的负标签。但由于Teacher模型的训练过程决定了负标签部分概率值都比较小，并且负标签的值越低，其信息就越不可靠。因此温度的选取需要进行实际实验的比较，本质上就是在下面两种情况之中取舍：
 - 当想从负标签中学到一些信息量的时候，温度应调高一些；
 - 当想减少负标签的干扰的时候，温度应调低一些；

总的来说，温度的选择和Student模型的大小有关，Student模型参数量比较小的时候，相对比较低的温度就可以了。因为参数量小的模型不能学到所有Teacher模型的知识，所以可以适当忽略掉。

七、代码执行

在这个例子中，将使用 DeBERTa-base作为 Teacher，使用其1、3、5、7、9、11层组合一个6层压缩版作为 Student。，首先要把Teacher已经在数据集上进行了微调，这使教师可以轻松地直接启动蒸馏训练作业。



7.1 教师微调

使用代码imdb_deberta_trainer.py训练教师模型，主要加载部分：

```
model_id = 'microsoft/deberta-v3-base'
tokenizer = AutoTokenizer.from_pretrained(model_id)

model = AutoModelForSequenceClassification.from_pretrained(model_id)
```

调用trainer.train()后，将model和tokenizer保存。

```
model.save_pretrained('deberta-v3-base-finetuned')
```

7.2 初始化学生模型

学生模型的初始化一般有两种方法：

- 使用teacher中的某些层次。
- 使用TinyBERT作为学生的基座，这种方法随意调一下，模型都很好。（后续请自行尝试这种方法）

首先，建立一个学生模型，随后再把教师模型的权重state_dict(), 用load_state_dict()方法加载过来：

```
def create_student(teacher_model):
    configuration = teacher_model.config.to_dict()
    configuration["num_hidden_layers"] //= 2
    configuration = DebertaV2Config.from_dict(configuration)
    student_model = type(teacher_model)(configuration)
    copy_deberta_weights(teacher_model, student_model)
    return student_model
```

注意最后一行的代码，拷贝teacher模型的权重到student模型，这里开始实现。

```
def copy_deberta_weights(teacher, student):
    if isinstance(teacher, DebertaV2Model) or
    type(teacher).__name__.startswith('DebertaV2For'):
        for teacher_part, student_part in zip(teacher.children(),
        student.children()):
            copy_deberta_weights(teacher_part, student_part)

    elif isinstance(teacher, DebertaV2Encoder):
        teacher_encoding_layers = [layer for layer in
        next(teacher.children())]
        student_encoding_layers = [layer for layer in
        next(student.children())]
        for i in range(len(student_encoding_layers)):
            student_encoding_layers[i].load_state_dict(teacher_encoding_layers[2*i].state_dict())

    else:
        student.load_state_dict(teacher.state_dict())
```

接下来创建student模型，并保存：

```
student = distill_deberta(deberta)
student.save_pretrained('deberta-v3-base-student')
```

也可以通过`print(student)`看看模型的结构是否符合。

7.3 蒸馏训练

通常，在使用 PyTorch 微调 Transformer 模型时，应该使用 `.Trainer` 类提供了一个 API，用于在大多数标准用例中使用 PyTorch 进行功能完整的训练。在示例中，因为需要传入两个模型`teacher`和`student`，并计算这两个模型的损失，因此不能直接使用 huggingface 的 `Trainer`。但是可以用继承的方法来创建一个 `DistillationTrainer`，并且只覆盖 `compute_loss` 方法来计算损失，把教师和学生之间的 KL 散度加上。除此之外，我们还需要继承 `TrainingArguments` 来包含蒸馏超参数，也就是温度 `temperature` 和两个损失的均衡稀疏 `alpha`。

```
class DistillationTrainingArguments(TrainingArguments):
    def __init__(self, *args, alpha=0.5, temperature=2.0, **kwargs):
        super().__init__(*args, **kwargs)

        self.alpha = alpha
        self.temperature = temperature

class DistillationTrainer(Trainer):
    def __init__(self, *args, teacher_model=None, **kwargs):
        super().__init__(*args, **kwargs)
        self.teacher = teacher_model
        # place teacher on same device as student
        self._move_model_to_device(self.teacher, self.model.device)
        self.teacher.eval()

    def compute_loss(self, model, inputs, return_outputs=False,
num_items_in_batch=None):
        # compute student output
        outputs_student = model(**inputs)
        student_loss = outputs_student.loss

        # compute teacher output
        with torch.no_grad():
            outputs_teacher = self.teacher(**inputs)

        # assert size
        assert outputs_student.logits.size() ==
outputs_teacher.logits.size()
```

```

        # Soften probabilities and compute distillation loss
        loss_function = nn.KLDivLoss(reduction="batchmean")
        loss_logits = (loss_function(
            F.log_softmax(outputs_student.logits / self.args.temperature,
dim=-1),
            F.softmax(outputs_teacher.logits / self.args.temperature,
dim=-1)) * (self.args.temperature ** 2))
        # Return weighted student loss
        loss = self.args.alpha * student_loss + (1. - self.args.alpha) *
loss_logits
        return (loss, outputs_student) if return_outputs else loss

```

其余数据集和评测函数请自行加载，这里开始加载teacher和student model

```

student_id = "deberta-v3-base-student"
teacher_id = "deberta-v3-base-finetuned"

# load model
teacher_model = AutoModelForSequenceClassification.from_pretrained(
    teacher_id,
)

# define student model
student_model = AutoModelForSequenceClassification.from_pretrained(
    student_id,
)

```

开始训练。

```

training_args = DistillationTrainingArguments(
    output_dir="checkpoint",
    num_train_epochs=3,
    per_device_train_batch_size=2,
    per_device_eval_batch_size=4,
    gradient_accumulation_steps=8,
    gradient_checkpointing=True,

    warmup_steps=500, # number of warmup steps for learning rate
scheduler
    weight_decay=0.01, # strength of weight decay
    logging_dir='./logs', # directory for storing logs
    logging_steps=100,

```



```

        save_strategy="no",
        evaluation_strategy="epoch",

        # distillation parameters
        alpha=0.5,
        temperature=4.0
    )

    trainer = DistillationTrainer(
        model=student_model,
        args=training_args,
        teacher_model=teacher_model,
        train_dataset=tokenized_train, # training dataset
        eval_dataset=tokenized_val, # evaluation dataset
        processing_class=tokenizer,
        data_collator=data_collator,
        compute_metrics=compute_metrics,
    )

    trainer.train()

```

最后评测一下student的性能如何？

```

prediction_outputs = trainer.predict(tokenized_test)
test_pred = np.argmax(prediction_outputs[0], axis=-1).flatten()
print(test_pred)

result_output = pd.DataFrame(data={"id": test["id"], "sentiment": test_pred})
result_output.to_csv("./result/deberta_base_student.csv", index=False,
quoting=3)
logging.info('result saved!')

```

那接下来，分别在SST-2和IMDB上尝试一下各种模型的蒸馏效果。这里特别注意，如果用TinyBERT直接作为学生模型，会有比较好的微调效果。分别尝试：

- 1、使用教师模型中的6层初始化学生模型。
- 2、使用TinyBERT模型初始化学生模型。
- 3、用Patient Knowledge Distillation尝试上面的方法。代码参考：
<https://github.com/intersun/PKD-for-BERT-Model-Compression>